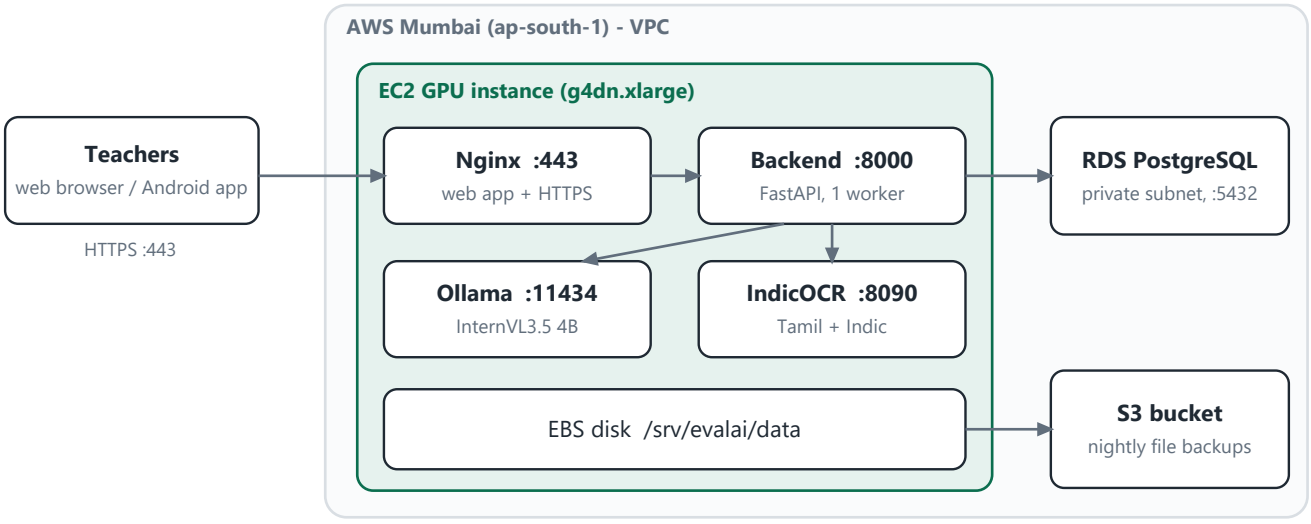
Eval AI on AWS

Deployment guide · Mumbai region (ap-south-1) · one GPU server + RDS PostgreSQL · 30 October 2026

What you end up with: the web app at **https://eval.yourschool.in** (use your own domain), the Android app pointing at the same address, all AI models running on one AWS GPU server, the database in managed PostgreSQL (RDS), and nightly backups in S3. Plan for **one working day** of setup once the GPU quota is approved.

1. What runs where

Part	Today (local PC)	On AWS
Ollama + InternVL3.5 4B	Port 11434; English reading and all grading	Same GPU server, localhost only
IndicOCR service	Port 8090; Tamil and other Indian languages	Same GPU server, localhost only
Backend (FastAPI)	Port 8000, started by start.bat	systemd service behind Nginx
Database	SQLite file data/adem.db	RDS PostgreSQL (DATABASE_URL)
Uploaded files and reports	data/exams, data/school	EBS disk at /srv/evalai/data, backed up to S3
Web frontend	Vite dev server :5173	Built once (npm run build), served by Nginx
Mobile app	Expo, phone types the PC's IP	Release APK using https://your-domain



Ports 8000, 8090, 11434 listen on 127.0.0.1 only; only 80/443 (and 22 from your IP) are open.

2. Before you start

1. **AWS account.** Turn on MFA for the root user and create a separate admin IAM user for daily work. Accounts with an Indian billing address are billed by AWS India (AISPL) in INR, plus 18% GST.
2. **GPU quota - do this first.** Service Quotas → Amazon EC2 → Running On-Demand G and VT instances → request **8** vCPUs in Mumbai. New accounts often start at 0. Approval can take from a few hours to a few days.

3. **Budget alert.** Billing → Budgets → create a monthly cost budget (for example US$550) with an email alert at 80%.
4. **Hugging Face.** Log in, open huggingface.co/bodhan-ai/indic-ocr and accept the licence, then create a **Read** token. IndicOCR needs it to download the model.
5. **Domain.** A domain or subdomain you control (for example eval.yourschool.in) and access to its DNS settings.
6. **Code in a private Git repository.** The Postgres support (DATABASE_URL, psycopg) is in your working copy but not committed yet. Commit it, push to a private GitHub repo, and add a read-only deploy key for the server.

3. Network and security groups

The default VPC in Mumbai is fine. Create two security groups (EC2 → Security Groups):

Security group	Inbound rule	Why
evalai-web	TCP 443 and 80 from 0.0.0.0/0	Web app and mobile app over HTTPS (80 only redirects)
evalai-web	TCP 22 from your IP/32	SSH for you only
evalai-db	TCP 5432 from security group evalai-web	Only the app server can reach the database

> Do **not** open 8000, 8090 or 11434. The backend, IndicOCR and Ollama listen on 127.0.0.1 and are reached only through Nginx or from the backend itself.

4. Create the PostgreSQL database (RDS)

1. RDS → Create database → **Standard create** → PostgreSQL **16** (or the newest 16/17 offered).
2. Template **Dev/Test**, deployment **Single-AZ**, instance **db.t4g.small** (2 vCPU, 2 GB).
3. DB instance identifier **evalai-db**, master username **evalai**, a strong password (keep it in your password manager).
4. Storage **gp3, 20 GB**, storage autoscaling on with a maximum of 100 GB.
5. Connectivity: same VPC as the server, **Public access: No**, security group **evalai-db**.
6. Additional configuration: initial database name **adem**, automated backups **7 days**, encryption on, **deletion protection on**.
7. When it shows Available, copy the **endpoint** (looks like evalai-db.xxxx.ap-south-1.rds.amazonaws.com).

The backend connection setting will be:

```
DATABASE_URL=postgresql+psycopg://evalai:PASSWORD@ENDPOINT:5432/adem?sslmode=require
```

If the password contains characters such as @ : / # %, URL-encode them (for example @ becomes %40). The backend creates all tables itself on first start.

5. Launch the GPU server (EC2)

1. EC2 → Launch instance, name **evalai**.
2. AMI: search for **Deep Learning Base OSS Nvidia Driver GPU AMI (Ubuntu 22.04)**. It already has the NVIDIA driver and CUDA, which saves the hardest part of the setup.
3. Instance type **g4dn.xlarge** (NVIDIA T4 16 GB, 4 vCPU, 16 GB RAM). Choose **g6.xlarge** (L4 24 GB) if you want to raise parallel jobs later.
4. Key pair: create one and keep the .pem file safe. Security group: **evalai-web**.
5. Storage: **120 GB gp3** root volume (AMI, PyTorch, the models and exam files).

6. IAM instance profile: a role with permission to write to your backup S3 bucket (step 10) and the managed policy **AmazonSSMManagedInstanceCore** (lets you open a browser shell without SSH).
7. After launch: EC2 → Elastic IPs → allocate one and associate it with the instance. In your DNS, add an **A record** eval.yourschool.in → that Elastic IP.

6. Install the software

Connect with `ssh -i evalai.pem ubuntu@ELASTIC_IP`, then:

```
sudo apt update && sudo apt -y upgrade
nvidia-smi                                  # must show the Tesla T4 (or L4)
sudo add-apt-repository -y ppa:deadsnakes/ppa
sudo apt install -y python3.11 python3.11-venv python3.11-dev git nginx \
                    postgresql-client certbot python3-certbot-nginx
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
sudo snap install aws-cli --classic

# a service user that owns the app and data
sudo useradd -r -m -d /srv/evalai -s /bin/bash evalai
sudo mkdir -p /srv/evalai/data && sudo chown -R evalai:evalai /srv/evalai
```

6.1 Ollama and InternVL

```
curl -fsSL https://ollama.com/install.sh | sh
sudo systemctl edit ollama
#   add these lines in the editor, then save:
#   [Service]
#   Environment="OLLAMA_HOST=127.0.0.1:11434"
#   Environment="OLLAMA_NUM_PARALLEL=2"
#   Environment="OLLAMA_KEEP_ALIVE=24h"
sudo systemctl restart ollama
ollama pull blaifa/InternVL3_5:4B
curl -s http://127.0.0.1:11434/api/tags      # model is listed
```

6.2 Get the code

```
sudo -iu evalai
git clone git@github.com:YOUR_ORG/eval_ai.git /srv/evalai/app
```

6.3 IndicOCR (as the evalai user)

```
cd /srv/evalai/app/indic_ocr_service
python3.11 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/hf auth login           # paste the Hugging Face Read token
.venv/bin/python install.py       # downloads ~1.9 GB model + matching PyTorch/CUDA
```

The README describes Windows commands. On Linux the steps are the same with `.venv/bin/` instead of `.venv\Scripts\`. install.py picks the CUDA build from nvidia-smi.

6.4 Backend (as the evalai user)

```
cd /srv/evalai/app/backend
python3.11 -m venv .venv
.venv/bin/pip install -r requirements.txt
cp .env.example .env
python3.11 -c "import secrets; print(secrets.token_urlsafe(48))"   # use as SECRET_KEY
nano .env
```

Set these values in `backend/.env` (leave the others as they are):

```
DATABASE_URL=postgresql+psycopg://evalai:PASSWORD@ENDPOINT:5432/adem?sslmode=require
DATA_DIR=/srv/evalai/data
SECRET_KEY=<the generated value>
CORS_ORIGINS=https://eval.yourschool.in
INTERNVL_API_URL=http://127.0.0.1:11434
INDIC_OCR_ENABLED=true
INDIC_OCR_URL=http://127.0.0.1:8090
INTERNVL_PARALLEL_JOBS=2
INDIC_PARALLEL_JOBS=2
```

6.5 Web frontend

```
cd /srv/evalai/app/frontend
npm ci && npm run build
exit                                          # back to the ubuntu user
sudo mkdir -p /var/www/evalai
sudo rsync -a --delete /srv/evalai/app/frontend/dist/ /var/www/evalai/
```

7. Run everything as services

Create `/etc/systemd/system/evalai-indicocr.service`:

```
[Unit]
Description=Eval AI - IndicOCR
After=network.target

[Service]
User=evalai
WorkingDirectory=/srv/evalai/app/indic_ocr_service
ExecStart=/srv/evalai/app/indic_ocr_service/.venv/bin/python -m uvicorn server:app \
          --host 127.0.0.1 --port 8090
Restart=always

[Install]
WantedBy=multi-user.target
```

Create `/etc/systemd/system/evalai-backend.service`:

```
[Unit]
Description=Eval AI - Backend
After=network-online.target ollama.service evalai-indicocr.service

[Service]
User=evalai
WorkingDirectory=/srv/evalai/app/backend
ExecStart=/srv/evalai/app/backend/.venv/bin/python -m uvicorn app.main:app \
          --host 127.0.0.1 --port 8000 --workers 1 --proxy-headers
Restart=always

[Install]
WantedBy=multi-user.target
```

> Keep **--workers 1**. The model job queue lives inside the backend process; several workers would each run their own queue and overload the GPU. Do not use --reload on the server.

```
sudo systemctl daemon-reload
sudo systemctl enable --now evalai-indicocr evalai-backend
curl -s http://127.0.0.1:8090/health
curl -s http://127.0.0.1:8000/api/health
journalctl -u evalai-backend -f              # live log; Ctrl+C to leave
```

8. Nginx and HTTPS

Create `/etc/nginx/sites-available/evalai`:

```
server {
    listen 80;
    server_name eval.yourschool.in;
    client_max_body_size 60M;              # MAX_UPLOAD_MB is 50

    root /var/www/evalai;
    index index.html;

    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 600s;            # AI requests can take minutes
        proxy_send_timeout 600s;
    }

    location / {
        try_files $uri /index.html;         # single-page app routes
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/evalai /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d eval.yourschool.in --redirect -m you@yourschool.in --agree-tos
```

Certbot adds the HTTPS settings and renews the certificate automatically. Open https://eval.yourschool.in and log in.

9. Mobile app

- Teachers can already type **https://eval.yourschool.in** in the server field on the login screen.
- For a proper release, make that address the default in mobile/src/api/client.ts and build a release APK with Expo EAS (`npx eas build -p android --profile production`). Android blocks plain http:// in release builds, so HTTPS is required.
- Publishing on Google Play needs a one-time US$25 developer account; sharing the APK directly does not.

10. Backups and monitoring

- **Database:** RDS automated backups (7 days, point-in-time restore) are already on from step 4.
- **Files:** create an S3 bucket such as **evalai-backups-yourschool** in Mumbai (block all public access, versioning on, lifecycle rule to delete old versions after 30 days). Then add a nightly copy:

```
sudo crontab -u evalai -e
# every night at 01:30
30 1 * * * aws s3 sync /srv/evalai/data s3://evalai-backups-yourschool/data --only-show-errors
```

- **Alerts:** CloudWatch alarm on the instance's StatusCheckFailed metric (email via SNS), plus the budget alert from step 2.
- **Logs:** `journalctl -u evalai-backend`, `journalctl -u evalai-indicocr`, `journalctl -u ollama`.
- **GPU:** `nvidia-smi` during a busy grading run. Raise INTERNVL_PARALLEL_JOBS and OLLAMA_NUM_PARALLEL together only if there is memory to spare.

11. Moving existing data (optional)

- **Files:** copy the local data/exams and data/school folders to /srv/evalai/data with scp or rsync, then `sudo chown -R evalai:evalai /srv/evalai/data`.
- **Database:** the SQLite data (data/adem.db) is not moved automatically. Either start with an empty database, or let the backend create the tables first and then copy the rows with a one-off script (not written yet).

12. Updating the app later

```
sudo -iu evalai
cd /srv/evalai/app && git pull
backend/.venv/bin/pip install -r backend/requirements.txt
cd frontend && npm ci && npm run build && exit
sudo rsync -a --delete /srv/evalai/app/frontend/dist/ /var/www/evalai/
sudo systemctl restart evalai-backend
```

Restart when no evaluations are running. The backend recovers interrupted jobs at startup, but restarting mid-run wastes GPU work.

13. Go-live checklist

- ■ GPU quota approved; budget alert set
- ■ nvidia-smi shows the GPU; ollama list shows blaifa/InternVL3_5:4B
- ■ /health on 8090 and /api/health on 8000 return OK from the server
- ■ https://your-domain loads with a valid padlock; http redirects to https
- ■ Ports 8000, 8090, 11434 and 5432 are not reachable from the internet
- ■ RDS: public access No, backups 7 days, deletion protection on
- ■ One complete trial exam in English and one in Tamil: upload paper, key, sheets, grade, finalize, download reports
- ■ Nightly S3 copy has run once (check the bucket the next morning)
- ■ backend/.env is not in Git; SECRET_KEY and database password are stored safely

14. Known risks and open items

- **Text length limits in PostgreSQL.** SQLite ignored VARCHAR limits; PostgreSQL rejects longer text. Columns fed by OCR or model output could overflow: review_reason and mapping_note (300 characters), raw_question_number (100), roll_number (50). A trial run will show whether this happens; trimming to the column size can be added in code.
- **Postgres is new for this app.** The automated tests run on SQLite. Run the full trial in the go-live checklist before real exams.
- **One server.** If the GPU instance stops, the whole app is offline until it restarts. That is acceptable for a first deployment; the database and file backups protect the data.
- **Student data.** Everything stays in the Mumbai region. Keep SSH limited to your IP, keep the bucket private, and give AWS console access only to people who need it.